

CERTIFICATE PROGRAM

AI Agents & Workflow Automation

A 12-week, plain-English course that takes you from "I know nothing about AI" to confidently understanding, designing, and certifying in autonomous AI agents — no coding required.

Format: Self-paced online · video lessons + visual slides + readings

Duration: 12 weeks across 4 phases

Level: Absolute beginner — friendly · concept-first · zero programming

Credential: Certificate of Completion (earned by passing the final exam)

START HERE

About this course

AI agents are software that can **think through a goal, decide what to do, take actions on their own, and check their own work** — instead of waiting for a human to click every button. They are quickly becoming the engine behind modern workflow automation: answering customers, researching markets, drafting reports, moving data between apps, and running multi-step business processes with little supervision.

This course is built for people who have **never written a line of code** and may feel that AI is intimidating or "too technical." Every concept is taught with plain language, real-world analogies, and simple visuals first. You will finish able to explain how agents work, recognize where they help a business, and design an automation on paper — the foundation you need before ever touching a tool.

Who this course is for

- Business owners, marketers, and managers who want to automate work but don't know where to start.
- Career-changers and students curious about AI who want a gentle, structured on-ramp.
- Operations and support teams who want to understand what's possible before hiring or buying tools.
- Anyone who learns best by **seeing visuals and reading clear explanations** rather than studying code.

What you'll be able to do by the end

- Explain in plain English how an AI agent perceives, reasons, and acts.
- Describe the core design patterns professionals use to make agents reliable.
- Recognize when one agent is enough and when a "team" of agents is better.
- Spot the risks of automation and know the guardrails that keep it safe.
- Design a complete, real-world automation workflow on paper — and pass the certification exam.

LOGISTICS

How the course works & how to earn your certificate

Each week includes short **video lessons**, a set of **visual slides** you can study at your own pace, and a **plain-language reading** that reinforces the key ideas. There is no homework you have to submit — you learn, you review, and you move on when you're ready.

Earning the Certificate of Completion

The certificate is awarded when you **pass the final exam** at the end of Week 12. The exam is multiple-choice and scenario-based, drawn entirely from the course material, and designed to confirm you understand the concepts — not to trick you.

Requirement	Detail
Final exam	~50 multiple-choice & scenario questions covering all 4 phases
Passing score	75% or higher
Retakes	Unlimited retakes; questions reshuffle each attempt
Time	Untimed — study at your own pace
Result	Pass → personalized Certificate of Completion issued automatically

HOW TO READ THIS CURRICULUM

Each week is a card showing its **learning objectives**, the **lessons** inside it, the **key concepts** explained simply, and special boxes: a green **Analogy** box (a real-world comparison) and a blue **On-screen visual** box (what the slide or animation should show). These boxes are the blueprint your video and slide content will be built from.

THE BIG PICTURE

Program map: 4 phases, 12 weeks

Phase	Weeks	Focus
1 • Foundations	1–3	What an agent is, how it senses-thinks-acts, how it uses tools, and how it breaks big goals into steps.
2 • Design Patterns	4–6	The repeatable "recipes" that make agents smarter and more reliable: reflection, chaining, routing, parallelization.
3 • Orchestration	7–9	Teams of agents working together, the frameworks that coordinate them, and how agents remember things.
4 • Production & Capstone	10–12	Testing, guardrails, watching what agents do, and designing a complete real-world system. Final exam.

The course deliberately moves from **simple single ideas** to **combined systems**. Nothing in a later week assumes knowledge you weren't given earlier, and every technical term is reintroduced with a plain definition the first time it appears in each phase.

PHASE 1

Foundational Concepts

Weeks 1–3 • The building blocks

Before automation, you need to understand what an AI agent actually *is*. This phase builds your mental model from the ground up: how an agent senses its situation, decides what to do, takes action, uses tools, and breaks a big goal into manageable steps.

LEARNING OBJECTIVES

- Define an "AI agent" in one plain sentence and explain how it differs from a normal chatbot or app.
- Describe the three-step loop every agent runs: **perceive** → **reason** → **act**.
- Understand what a Large Language Model (LLM) is and why it's the "brain" inside most agents.
- Recognize the ReAct idea — that an agent alternates between *thinking* and *doing*.

LESSONS

- **1.1 — "Software that does, not just answers."** The difference between a tool you operate and an agent that operates on your behalf.
- **1.2 — Meet the LLM.** A friendly, no-math explanation of what a language model is: a very well-read assistant that predicts the most sensible next words.
- **1.3 — The agent loop.** Perceive (take in the situation) → Reason (think about what to do) → Act (do something in the real world) → repeat until the goal is met.
- **1.4 — ReAct: think, then act, then think again.** Why letting the agent "reason out loud" between actions makes it far more reliable.

KEY CONCEPTS, EXPLAINED SIMPLY

Agent: a program that is given a goal and figures out the steps to reach it on its own, taking actions along the way. **LLM:** the language "brain" that reads instructions and produces sensible responses.

The loop: agents don't act once — they repeat sense-think-act until done, just like a person working through a task.

ANALOGY

An agent is like a **capable new assistant**. You don't tell them which keys to press — you tell them the *goal* ("book me a sensible flight to Chicago"). They look at the options (perceive), weigh them (reason), and book one (act). A regular app is a calculator: it only does the exact button you press.

ON-SCREEN VISUAL

An animated circular diagram: three arrows forming a loop labeled **Perceive** → **Reason** → **Act**, with a small brain icon in the center labeled "LLM." Each loop highlights as it's described.

BY THE END OF WEEK 1 YOU CAN...

Explain to a friend, with no jargon, what an AI agent is and the simple loop it follows.

LEARNING OBJECTIVES

- Explain why an LLM on its own can only talk — and what "tools" let it actually *do*.
- Understand "tool calling": how an agent decides to use an outside function, then uses the result.
- Know, in plain terms, what an API is and why it's how apps talk to each other.
- List common agent tools: web search, calculators, databases, email/calendar, and other apps.

LESSONS

- **2.1 — Why a brain needs hands.** An LLM alone can't check today's weather or send an email; tools connect it to the real world.
- **2.2 — What "tool calling" means.** The agent recognizes "I need a fact I don't have," picks the right tool, uses it, and reads the answer back.
- **2.3 — APIs in plain English.** An API is a "service window" one app opens so other apps can request things in a predictable way.
- **2.4 — A tour of common tools.** Web search, databases (the agent's filing cabinet), code execution (a built-in calculator), and connections to apps like Gmail, Slack, or a CRM.

KEY CONCEPTS, EXPLAINED SIMPLY

Tool: any outside ability the agent can call on — search, a database, a calculator, another app. **Tool calling:** the moment the agent says "I'll use this tool now," runs it, and uses what comes back. **API:** the standardized "ordering window" that lets the agent request something from another service and get a tidy answer.

ANALOGY

The LLM is a **brilliant person locked in a quiet room**. They can reason about anything, but to learn today's news or send a message they need a **phone and a computer**. Tools are those devices. An API is the **drive-through window**: you say your order in the expected format, and the kitchen hands back exactly what you asked for.

ON-SCREEN VISUAL

The brain icon from Week 1 grows "hands," each reaching toward a labeled tool icon: a magnifying glass (search), a filing cabinet (database), a calculator, and an envelope (email). Arrows show information flowing back into the brain.

BY THE END OF WEEK 2 YOU CAN...

Explain why agents need tools, and name several tools an agent might use to complete a real task.

LEARNING OBJECTIVES

- Understand why agents break a large, vague goal into smaller, ordered sub-tasks.
- Describe simple planning: list the steps, then do them in sensible order.
- See how planning + tools + the agent loop combine into a complete simple agent.
- Practice (on paper) breaking a real goal like "plan a product launch email" into steps.

LESSONS

- **3.1 — Why planning matters.** "Grow my business" is too big to act on; "write 3 launch emails this week" is doable. Agents need the smaller version.
- **3.2 — Decomposition.** The skill of splitting one big goal into a checklist of sub-tasks the agent can tackle one at a time.
- **3.3 — Putting it together.** A walkthrough of one simple agent that plans, uses a tool, checks the result, and moves to the next step.
- **3.4 — Phase 1 recap.** Perceive-reason-act + tools + planning = your complete foundation.

KEY CONCEPTS, EXPLAINED SIMPLY

Planning: deciding the steps before starting. **Decomposition:** breaking a big goal into bite-size sub-tasks. **Sub-task:** one small, clearly doable piece of the bigger goal.

ANALOGY

Planning is a **recipe**. "Make dinner" is overwhelming; a recipe turns it into "chop onions, heat pan, add oil..." — small steps in the right order. An agent writes itself a recipe before it starts cooking.

ON-SCREEN VISUAL

A single large box labeled "GOAL: launch our new service" splits with branching arrows into a numbered checklist of 4–5 smaller boxes (research audience → draft email → design graphic → schedule → review), each with a checkbox.

BY THE END OF WEEK 3 YOU CAN...

Take any everyday business goal and break it into a clear, ordered list of sub-tasks — the way an agent does.

PHASE 2

Core Design Patterns

Weeks 4–6 • The repeatable recipes

Professionals don't reinvent agents each time — they reuse proven "patterns." This phase teaches the four most important ones in plain language: how agents check their own work, pass results down a chain, send tasks to the right specialist, and work on several things at once.

LEARNING OBJECTIVES

- Explain why an agent's *first* answer is often not its *best* answer.
- Describe "reflection": the agent reviews its own output, finds flaws, and improves it.
- Recognize the draft → critique → revise cycle and why it boosts quality.
- Identify business tasks where self-checking matters most (reports, emails, code, summaries).

LESSONS

- **4.1 — The first draft problem.** Like people, agents produce rough first attempts; quality comes from revision.
- **4.2 — How reflection works.** The agent (or a second "critic" agent) asks "what's wrong with this?" and rewrites accordingly.
- **4.3 — When to use it.** High-stakes or customer-facing output deserves a self-review pass; quick throwaway tasks may not.

KEY CONCEPTS, EXPLAINED SIMPLY

Reflection: the agent grades and improves its own answer before showing it to you. **Critic:** the reviewing role — sometimes the same agent wearing a different "hat," sometimes a second agent.

Iteration: repeating draft-and-improve until it's good enough.

ANALOGY

Reflection is **proofreading your own email before hitting send**. The first version has typos and an awkward sentence; you reread it, fix the issues, and only then send. The agent does exactly this, automatically.

ON-SCREEN VISUAL

A document icon labeled "Draft 1" flows into a magnifying-glass "Critique" step (red marks appear), then into "Draft 2" (clean, green check). A loop arrow shows it can repeat.

BY THE END OF WEEK 4 YOU CAN...

Explain reflection and identify which of your own work tasks would benefit from an agent that checks itself.

LEARNING OBJECTIVES

- Define "prompt chaining": linking steps so each one's output feeds the next.
- Understand why splitting a job into linked steps beats asking for everything at once.
- Trace a simple chain from start to finish (e.g., research → outline → draft → polish).
- Recognize where one weak link can affect the whole chain.

LESSONS

- **5.1 — One big ask vs. a chain of small ones.** Why breaking a request into ordered steps produces cleaner results.
- **5.2 — Output becomes input.** The heart of chaining: step 2 starts from exactly what step 1 produced.
- **5.3 — A worked example.** A content-creation chain: gather facts → make outline → write draft → tighten and format.
- **5.4 — Quality control in a chain.** Why a mistake early on flows downstream, and how checks between steps help.

KEY CONCEPTS, EXPLAINED SIMPLY

Prompt chaining: a sequence of steps where each hands its result to the next. **Step / link:** one stage in that sequence. **Hand-off:** the moment one step's output becomes the next step's starting material.

ANALOGY

Chaining is a **factory assembly line**. Station 1 attaches the wheels and passes the car to Station 2, which adds doors, and so on. Each station does one job well and hands the work along, rather than one worker building the whole car alone.

ON-SCREEN VISUAL

A horizontal conveyor belt with 4 stations (Research → Outline → Draft → Polish). A "work item" slides left to right, visibly growing more complete at each station.

BY THE END OF WEEK 5 YOU CAN...

Sketch a multi-step chain for a real task and explain how each step feeds the next.

Routing & parallelization — the right specialist, and doing things at once

LEARNING OBJECTIVES

- Explain "routing": sorting an incoming task and sending it to the best-suited specialist.
- Explain "parallelization": running several steps at the same time, then combining results.
- Tell the difference between work that must be sequential and work that can happen simultaneously.
- See how routing + parallelization make automations faster and more accurate.

LESSONS

- **6.1 — Routing: the smart receptionist.** An incoming request is classified ("this is a billing question") and sent to the agent best at handling it.
- **6.2 — Parallelization: many hands at once.** When sub-tasks don't depend on each other, run them together to save time, then merge the findings.
- **6.3 — Sequential vs. parallel.** Some steps must wait their turn (chaining); others can run side by side. Knowing which is a key design skill.
- **6.4 — Phase 2 recap.** Reflection, chaining, routing, parallelization — the four recipes you'll combine later.

KEY CONCEPTS, EXPLAINED SIMPLY

Routing: deciding where a task should go based on what kind of task it is. **Specialist:** an agent tuned for one type of job. **Parallelization:** doing independent tasks at the same time. **Synthesis:** combining several results into one answer.

ANALOGY

Routing is a **hospital triage nurse** who sends each patient to the right department. Parallelization is a **kitchen during a dinner rush**: the salad, the steak, and the dessert are all prepared at once by different cooks, then plated together — far faster than making them one after another.

ON-SCREEN VISUAL

Left side: an arrow labeled "incoming task" hits a diamond "Router" that branches to three specialist icons (Sales, Support, Billing). Right side: one task splits into three lanes running simultaneously, then merges into a single "combined result" box.

BY THE END OF WEEK 6 YOU CAN...

Decide whether a given workflow needs routing, parallelization, or both — and explain why.

PHASE 3

Advanced Orchestration

Weeks 7–9 • Teams, frameworks & memory

Now we scale up. This phase shows how multiple agents work together as a coordinated team, introduces the industry "control panels" (frameworks) that manage them — explained without any code — and reveals how agents remember things both short-term and long-term.

LEARNING OBJECTIVES

- Explain why several specialized agents often beat one agent trying to do everything.
- Describe the orchestrator–worker model: a "manager" agent delegating to "worker" agents.
- Understand how agents pass information and hand off tasks to one another.
- Recognize real business examples of agent teams.

LESSONS

- **7.1 — Why a team?** A specialist who does one thing well usually beats a generalist juggling everything — the same is true for agents.
- **7.2 — The orchestrator and the workers.** A manager agent breaks down the goal, assigns pieces to worker agents, and assembles their results.
- **7.3 — How agents talk to each other.** Hand-offs, shared instructions, and reporting results back up to the manager.
- **7.4 — Real examples.** A research team (one finds sources, one summarizes, one fact-checks) and a content team (writer + editor + designer).

KEY CONCEPTS, EXPLAINED SIMPLY

Multi-agent system: several agents working together. **Orchestrator:** the manager agent that delegates and coordinates. **Worker:** a specialist agent that handles one assigned part. **Delegation:** handing a sub-task to the agent best suited for it.

ANALOGY

An agent team is a **small company**. The orchestrator is the project manager who understands the whole goal and assigns tasks; the workers are employees who each own their specialty. The manager doesn't do every job — they make sure the right person does each one and that the pieces fit together.

ON-SCREEN VISUAL

An org-chart layout: a single "Orchestrator" box at the top with arrows down to three "Worker" boxes (Researcher, Writer, Fact-checker). Dotted arrows show results flowing back up and combining.

BY THE END OF WEEK 7 YOU CAN...

Design a simple agent "org chart" for a real task, naming the manager and the specialists.

Agentic frameworks — the control panels (LangGraph & CrewAI, no code)

LEARNING OBJECTIVES

- Understand what a "framework" is and why builders use one instead of starting from scratch.
- Grasp, conceptually, what LangGraph does: maps an agent's workflow as a flowchart of steps and decisions.
- Grasp, conceptually, what CrewAI does: organizes agents into a crew with defined roles and tasks.
- Understand "stateful workflows" — the system remembering where it is in a multi-step process.

LESSONS

- **8.1 — What a framework is.** A ready-made set of building blocks and rules that handles the plumbing so builders focus on the workflow.
- **8.2 — LangGraph, conceptually.** Think of your agent's process as a *flowchart*: boxes for steps, diamonds for decisions, arrows for what happens next. LangGraph is a tool for drawing and running that flowchart.
- **8.3 — CrewAI, conceptually.** A way to define a "crew" of agents by giving each a role, a goal, and tasks — then letting them collaborate. (Connects directly to Week 7's team idea.)
- **8.4 — "Stateful" in plain English.** The system keeps track of progress so it can pause, resume, and follow long multi-step paths without losing its place.

KEY CONCEPTS, EXPLAINED SIMPLY

Framework: a pre-built toolkit that saves builders from starting at zero. **LangGraph:** a tool for laying out an agent's workflow as a runnable flowchart. **CrewAI:** a tool for organizing a team of role-based agents. **State:** the system's memory of "where we are" in a process.

ANALOGY

A framework is **pre-fab construction**. Instead of milling your own lumber, you get standardized walls and beams that snap together — you still design the house, but you don't reinvent every nail. LangGraph hands you a **flowchart kit**; CrewAI hands you a **team-roster kit**.

ON-SCREEN VISUAL

Split screen. Left: a clean flowchart (Start → step → decision diamond → two branches → End) labeled "LangGraph: your workflow as a map." Right: three labeled role cards (Researcher / Analyst / Writer) under a "Crew" header labeled "CrewAI: your agents as a team." No code shown — just the diagrams.

BY THE END OF WEEK 8 YOU CAN...

Explain what frameworks do and tell, at a high level, when a flowchart-style approach vs. a team-of-roles approach fits a problem.

LEARNING OBJECTIVES

- Explain why memory matters: without it, an agent forgets everything between steps.
- Tell the difference between short-term memory (the current conversation) and long-term memory (a permanent store).
- Understand a "vector database" in plain terms: a smart filing system that finds things by *meaning*, not exact words.
- See how memory makes agents feel consistent and personalized.

LESSONS

- **9.1 — Why agents forget.** By default, an LLM only "remembers" what's in front of it right now; memory systems fix this.
- **9.2 — Short-term (in-context) memory.** The agent's working notepad for the current task or conversation.
- **9.3 — Long-term memory & vector databases.** A storehouse the agent can search later — and it retrieves by meaning, so "cancel my plan" finds notes about "ending a subscription."
- **9.4 — Phase 3 recap.** Teams + frameworks + memory = agents that can handle big, ongoing, real-world work.

KEY CONCEPTS, EXPLAINED SIMPLY

Short-term memory: what the agent is holding in mind right now. **Long-term memory:** a saved store it can look things up in later. **Vector database:** a filing system that groups information by meaning so the agent can find related ideas even when the wording differs.

ANALOGY

Short-term memory is the **sticky note on your desk** for today's task. Long-term memory is the **filing cabinet** behind you. A vector database is a **filing cabinet organized by topic, not alphabet** — ask for "money owed to us" and it pulls the "unpaid invoices" folder, because it understands they mean the same thing.

ON-SCREEN VISUAL

A desk scene: a small sticky note labeled "short-term: this conversation" beside a large filing cabinet labeled "long-term: vector database." A search bar typing "money we're owed" highlights a folder labeled "unpaid invoices," with a dotted "meaning match" line connecting them.

BY THE END OF WEEK 9 YOU CAN...

Explain why memory matters and describe, simply, how an agent stores and recalls information.

PHASE 4

Production, Evals & Capstone

Weeks 10–12 · Making it safe, real, and certified

A working agent and a *trustworthy* agent are different things. This final phase covers how to test agents, set up guardrails, watch what they actually do, and design a complete real-world system — finishing with the certification exam.

LEARNING OBJECTIVES

- Understand why multi-step agents fail in new ways, and why testing them is essential.
- Explain "error analysis": calmly studying what went wrong to make it better.
- Define "guardrails": rules and limits that keep an agent safe and on-task.
- Recognize common safeguards: human approval, limits on actions, and graceful recovery.

LESSONS

- **10.1 — Why agents fail.** With many steps, a small early mistake can snowball; one tool being down can stall the whole task.
- **10.2 — Error analysis.** The disciplined habit of reviewing failures, finding the pattern, and fixing the root cause.
- **10.3 — Guardrails.** Boundaries like "never send money without approval," "stay on topic," and "stop and ask a human if unsure."
- **10.4 — Graceful failure.** A good agent that hits a problem retries sensibly or hands off to a person — it doesn't crash silently or make things up.

KEY CONCEPTS, EXPLAINED SIMPLY

Error analysis: studying mistakes to improve. **Guardrail:** a built-in rule that limits what the agent may do. **Human-in-the-loop:** a checkpoint where a person approves an important action. **Graceful failure:** failing safely — retry or escalate, never pretend.

ANALOGY

Guardrails are exactly what they sound like — the **rails on a mountain road**. They don't drive the car, but they stop a small slip from becoming a disaster. A human-in-the-loop is the **"are you sure?" confirmation** before a big, irreversible action.

ON-SCREEN VISUAL

A winding road with guardrails; a car (the agent) approaches a fork. A "stop sign" labeled "needs human approval" appears before a high-risk branch (e.g., "send payment"), while a low-risk branch (e.g., "draft summary") proceeds freely.

BY THE END OF WEEK 10 YOU CAN...

List the guardrails a real automation should have and explain what "failing gracefully" looks like.

LEARNING OBJECTIVES

- Explain "observability": the ability to see every step an agent took and why.
- Understand why a visible "trail" is essential for trust, debugging, and improvement.
- Read a simple step-by-step agent trace and spot where something went wrong.
- Connect observability to the error analysis and guardrails from Week 10.

LESSONS

- **11.1 — The black-box problem.** When an agent gives a wrong answer, you need to see *how* it got there — not just the result.
- **11.2 — Traces & logs.** The recorded play-by-play of each thought, tool use, and decision the agent made.
- **11.3 — Reading a trace.** A guided walkthrough finding the exact step where an agent went off course.
- **11.4 — Why it builds trust.** Being able to inspect an agent's reasoning is what makes businesses comfortable relying on it.

KEY CONCEPTS, EXPLAINED SIMPLY

Observability: being able to see and understand what the agent did internally. **Trace:** the recorded sequence of the agent's steps. **Log:** a written record of events. **Debugging:** using that record to find and fix problems.

ANALOGY

Observability is a **flight recorder (black box)** for your agent. If something goes wrong, you don't guess — you replay the recording and see the exact moment and reason. It's also like a **receipt that itemizes every step**, not just the final total.

ON-SCREEN VISUAL

A vertical timeline/trace: each row is a step (Thought → Tool used → Result), with timestamps. One row is highlighted red — "here's where it picked the wrong data" — showing how a trace pinpoints failure.

BY THE END OF WEEK 11 YOU CAN...

Read a basic agent trace and identify where and why a multi-step task went wrong.

LEARNING OBJECTIVES

- Combine everything: loop, tools, planning, patterns, teams, memory, guardrails, observability.
- Design (on paper / as a diagram) a complete real-world automation from goal to guardrails.
- Walk through two worked example systems end to end.
- Review for and pass the final certification exam.

LESSONS

- **12.1 — The capstone blueprint.** A simple 6-part template: Goal → Steps → Tools → Pattern(s) → Memory → Guardrails & checks.
- **12.2 — Worked example: automated research assistant.** Takes a topic, plans research, searches, summarizes, self-reflects, and returns a sourced brief.
- **12.3 — Worked example: project-manager agent.** Routes incoming requests, delegates to worker agents, remembers project context, and asks a human before high-impact actions.
- **12.4 — Exam review & the final exam.** A guided recap of all four phases, then the certification exam.

ANALOGY

The capstone is your **driving test** after the lessons. You won't build an engine — you'll show you can confidently plan a safe, sensible "route" using everything you've learned. Designing the system on paper proves you truly understand it.

ON-SCREEN VISUAL

The 6-part capstone template as a fillable diagram, then an animated full example (the research assistant) lighting up each part in sequence — visibly reusing icons from earlier weeks (loop, tools, reflection, memory, guardrail) so learners recognize the payoff.

BY THE END OF WEEK 12 YOU CAN...

Design a complete agent-based automation on paper and pass the certification exam to earn your Certificate of Completion.

CERTIFICATION

Final exam blueprint

The final exam confirms you understand the concepts taught across all 12 weeks. It is multiple-choice and scenario-based, untimed, and can be retaken until passed. A score of **75%** earns the Certificate of Completion.

Section	Drawn from	Approx. share
Foundations: the loop, LLMs, tools, planning	Phase 1 (Wks 1–3)	25%
Design patterns: reflection, chaining, routing, parallelization	Phase 2 (Wks 4–6)	25%
Orchestration: agent teams, frameworks, memory	Phase 3 (Wks 7–9)	25%
Production: error analysis, guardrails, observability, system design	Phase 4 (Wks 10–12)	25%

Question styles (all beginner-appropriate)

- **Definitions:** "Which of these best describes a vector database?"
- **Match the analogy:** "The 'triage nurse' analogy describes which pattern?"
- **Scenario:** "A business wants invoices read, summarized, and flagged for approval before payment. Which guardrail is essential?"
- **Design choice:** "These three tasks don't depend on each other — sequential chaining or parallelization?"

BUILD NOTE FOR NUMBER1 DIGITAL MARKETING

This blueprint maps one-to-one onto the weeks, so a question bank can be written straight from each week's "Key concepts" and "Analogy" boxes. Aim for ~12–13 questions per phase (~50 total) and pull randomly so each retake reshuffles.

REFERENCE

Plain-English glossary

A quick reference learners can keep open. Every term is defined the way it's introduced in the course.

Term	Plain-English meaning
AI agent	Software given a goal that figures out and carries out the steps on its own.
LLM (Large Language Model)	The language "brain" that reads instructions and produces sensible responses.
Perceive–reason–act loop	The repeating cycle an agent runs: sense the situation, think, do something, repeat.
ReAct	The idea of an agent alternating between reasoning and taking action.
Tool	An outside ability the agent can use — search, a database, a calculator, another app.
API	A standardized "service window" that lets apps request things from each other.
Planning / decomposition	Breaking a big goal into a clear, ordered list of smaller sub-tasks.

Reflection	The agent reviewing and improving its own work before finalizing it.
Prompt chaining	Linking steps so each step's output becomes the next step's input.
Routing	Sorting an incoming task and sending it to the best-suited specialist agent.
Parallelization	Running independent tasks at the same time, then combining the results.
Multi-agent system	Several specialized agents working together as a team.
Orchestrator / worker	A "manager" agent that delegates tasks to specialist "worker" agents.
Framework	A pre-built toolkit (e.g., LangGraph, CrewAI) that saves builders from starting at zero.
State / stateful	The system remembering where it is in a multi-step process.
Short-term memory	What the agent is holding in mind for the current task or conversation.
Long-term memory	A saved store the agent can search later for past information.
Vector database	A filing system that finds information by meaning, not exact wording.
Guardrail	A built-in rule that limits what the agent is allowed to do.
Human-in-the-loop	A checkpoint where a person approves an important action.
Observability / trace	The ability to see, step by step, what the agent actually did and why.

Next step in our build

With this curriculum approved, the natural next move is to build **Week 1 in full** — the narration script (for your AI video tool) plus a designed slide deck — then continue module by module. Each week's slides and script come straight from the objectives, key concepts, analogy, and visual notes you see above.